

AI Hardware Architecture

Author: Nitin Sarangdhar

Affiliation: SemiX Visiting Professor of practice | IIT Bombay

Date: March 2026

Lab: Implementing the XOR Logic Gate from Scratch

Description: This notebook demonstrates the fundamental Forward and Backward pass mechanics of a (2, 2, 1) Deep Neural Network (DNN) using Stochastic Gradient Descent (SGD).

```
1 import numpy as np
2
3 # Training Data (4 possible combinations of 2 inputs)
4 # X shape: (4 samples, 2 features)
5 X_train = np.array([
6     [0, 0],
7     [0, 1],
8     [1, 0],
9     [1, 1]
10 ])
11
12 # Target Labels (The 'Correct' answers for XOR)
13 # y shape: (4 samples, 1 output)
14 y_train = np.array([
15     [0],
16     [1],
17     [1],
18     [0]
19 ])
20
21 print("Dataset defined.")
22 print("Inputs (X):\n", X_train)
23 print("Targets (y):\n", y_train)
```

```
Dataset defined.
Inputs (X):
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
Targets (y):
[[0]
 [1]
 [1]
 [0]]
```

Define the DNN. It has 3 layers. The input layer has 2 input neurons, the hidden layer has two neurons and the output layer has 1 neuron. In addition there is a bias term.

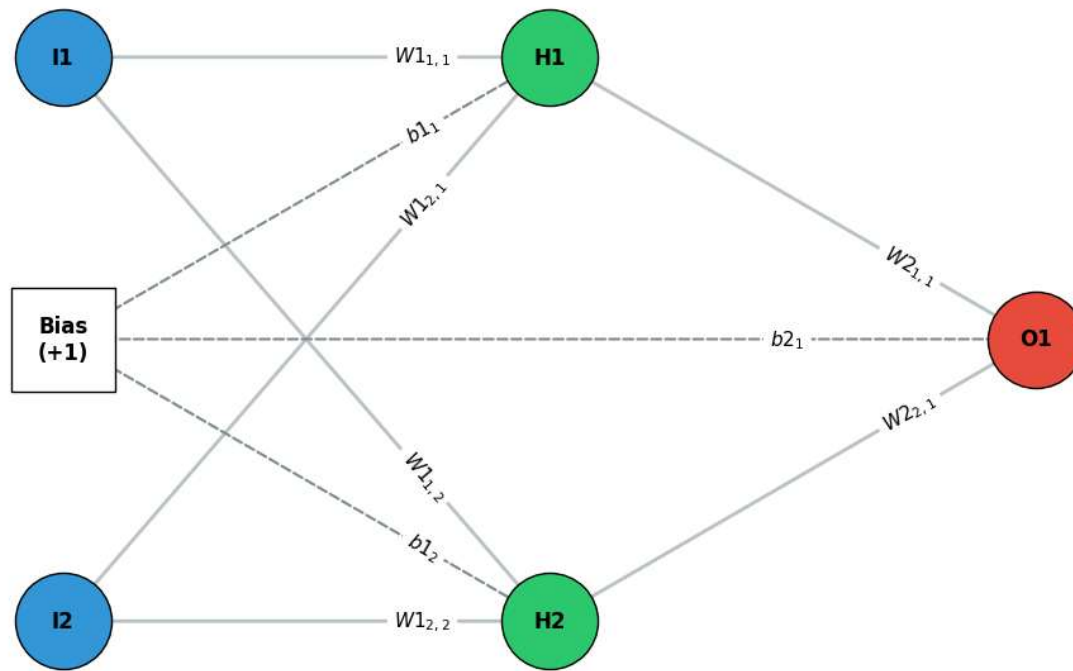
```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 def draw_minimal_xor_dnn():
5     # Minimal XOR Architecture: 2 Inputs, 2 Hidden, 1 Output
6     N_INPUT = 2
7     N_HIDDEN = 2
8     N_OUTPUT = 1
9
10    G = nx.DiGraph()
11    pos = {}
12    Y_SPREAD = 2.0 # Increased spread for a cleaner "minimal" look
13
14    # --- 1. Define Node Positions ---
15    # Input Layer (x=0)
16    pos[0] = (0, Y_SPREAD/2) # I1
17    pos[1] = (0, -Y_SPREAD/2) # I2
18    G.add_node(0, label='I1')
19    G.add_node(1, label='I2')
20
21    # BIAS NODE: Vertically centered in the Input column (x=0, y=0)
22    # Aligns perfectly with the gap between H1 and H2
23    BIAS_NODE_ID = "BIAS"
24    G.add_node(BIAS_NODE_ID, label=BIAS_NODE_ID)
```

```

24 pos[BIAS_NODE_ID] = (0, 0)
25 G.add_node(BIAS_NODE_ID, label='Bias\n(+1)')
26
27 # Hidden Layer (x=1)
28 # H1 at +1.0, H2 at -1.0
29 for i in range(N_HIDDEN):
30     node_id = N_INPUT + i
31     pos[node_id] = (1, Y_SPREAD/2 if i == 0 else -Y_SPREAD/2)
32     G.add_node(node_id, label=f'H{i+1}')
33
34 # Output Layer (x=2)
35 pos[4] = (2, 0) # O1
36 G.add_node(4, label='O1')
37
38 # --- 2. Define Edges and Labels (w_dest,source) ---
39 edge_labels = {}
40 primary_edges = []
41 bias_edges = []
42
43 # Layer 1 Weights: Input -> Hidden
44 for n in range(N_INPUT):
45     for m in range(N_HIDDEN):
46         u, v = n, N_INPUT + m
47         G.add_edge(u, v)
48         primary_edges.append((u, v))
49         edge_labels[(u, v)] = f'$w1_{{{n+1},{m+1}}}$'
50
51 # Layer 2 Weights: Hidden -> Output
52 for n in range(N_HIDDEN):
53     for m in range(N_OUTPUT):
54         u, v = N_INPUT + n, N_INPUT + N_HIDDEN + m
55         G.add_edge(u, v)
56         primary_edges.append((u, v))
57         edge_labels[(u, v)] = f'$w2_{{{n+1},{m+1}}}$'
58
59 # Bias Weights: Bias -> Hidden/Output
60 for m in range(N_HIDDEN):
61     u, v = BIAS_NODE_ID, N_INPUT + m
62     G.add_edge(u, v)
63     bias_edges.append((u, v))
64     edge_labels[(u, v)] = f'$b1_{{{m+1}}}$'
65
66 for m in range(N_OUTPUT):
67     u, v = BIAS_NODE_ID, N_INPUT + N_HIDDEN + m
68     G.add_edge(u, v)
69     bias_edges.append((u, v))
70     edge_labels[(u, v)] = f'$b2_{{{m+1}}}$'
71
72 # --- 3. Rendering ---
73 fig, ax = plt.subplots(figsize=(12, 7))
74
75 # Nodes
76 nx.draw_networkx_nodes(G, pos, nodelist=[0, 1], node_color='#3498db', node_size=3000, edgecolors='black')
77 nx.draw_networkx_nodes(G, pos, nodelist=[2, 3], node_color='#2ecc71', node_size=3000, edgecolors='black')
78 nx.draw_networkx_nodes(G, pos, nodelist=[4], node_color='#e74c3c', node_size=3000, edgecolors='black')
79 nx.draw_networkx_nodes(G, pos, nodelist=[BIAS_NODE_ID], node_color='white', node_shape='s', node_size=3500, edgecolors='black')
80
81 # Labels
82 nx.draw_networkx_labels(G, pos, labels=nx.get_node_attributes(G, 'label'), font_size=12, font_weight='bold')
83
84 # Edges
85 nx.draw_networkx_edges(G, pos, edgelist=primary_edges, edge_color='#bdc3c7', width=2, arrowsize=25)
86 nx.draw_networkx_edges(G, pos, edgelist=bias_edges, edge_color='#7f8c8d', style='dashed', width=1.5, arrowsize=20)
87
88 # Edge Labels
89 nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=11, label_pos=0.75)
90
91 plt.title("Minimal XOR Network Architecture (2, 2, 1)", fontsize=18, pad=20)
92 plt.axis('off')
93 plt.show()
94
95 draw_minimal_xor_dnn()

```

Minimal XOR Network Architecture (2, 2, 1)



Each neuron in the previous layer is connected to the neuron in the next layer with a weight. The bias is also connected to each neuron with a weight. Initialize the weights to random values

```

1 import numpy as np
2
3 # Set seed for reproducibility
4 np.random.seed(42)
5
6 # --- Layer 1: Input (2) -> Hidden (2) ---
7 # W1 shape: (destination_neurons, source_neurons) -> (2, 2)
8 W1 = np.random.randn(2, 2)
9 # b1 shape: (destination_neurons, 1) -> (2, 1)
10 b1 = np.random.randn(2, 1)
11
12 # --- Layer 2: Hidden (2) -> Output (1) ---
13 # W2 shape: (destination_neurons, source_neurons) -> (1, 2)
14 W2 = np.random.randn(1, 2)
15 # b2 shape: (destination_neurons, 1) -> (1, 1)
16 b2 = np.random.randn(1, 1)
17
18 # --- Display for the Class ---
19 print("-" * 40)
20 print("LAYER 1 INITIALIZATION (Input to Hidden)")
21 print("-" * 40)
22 print(f"Weights Matrix W1 (2x2):\n{W1}")
23 print(f"\nBias Vector b1 (2x1):\n{b1}")
24
25 print("\n" + "-" * 40)
26 print("LAYER 2 INITIALIZATION (Hidden to Output)")
27 print("-" * 40)
28 print(f"Weights Matrix W2 (1x2):\n{W2}")
29 print(f"\nBias Vector b2 (1x1):\n{b2}")
30 print("-" * 40)

```

```

-----
LAYER 1 INITIALIZATION (Input to Hidden)
-----

```

```

Weights Matrix W1 (2x2):
[[ 0.49671415 -0.1382643 ]
 [ 0.64768854  1.52302986]]

```

```

Bias Vector b1 (2x1):
[[-0.23415337]
 [-0.23413696]]

```

```

-----
LAYER 2 INITIALIZATION (Hidden to Output)
-----

```

```

-----
Weights Matrix W2 (1x2):
[[1.57921282 0.76743473]]

Bias Vector b2 (1x1):
[[-0.46947439]]
-----

```

Start training the network. In this example we are training the network through 1 episode. Each input is passed through the forward pass logic to calculate the predicted output. The predicted output is compared against the actual output to calculate the error. The error is used to apply a correction to the weights. The whole process is repeated until all inputs go through the same process.

```

1 import numpy as np
2
3 # 1. Setup Data and Network
4 np.random.seed(42)
5 X_train = np.array([[0,0], [0,1], [1,0], [1,1]])
6 y_train = np.array([[0], [1], [1], [0]])
7
8 # Initialize (2, 2, 1) Network
9 W1 = np.random.randn(2, 2)
10 b1 = np.random.randn(2, 1)
11 W2 = np.random.randn(1, 2)
12 b2 = np.random.randn(1, 1)
13
14 learning_rate = 0.5
15
16 def sigmoid(x):
17     return 1 / (1 + np.exp(-x))
18
19 def sigmoid_derivative(a):
20     # Derivative of sigmoid using the activation 'a'
21     return a * (1 - a)
22
23 # 2. Training Loop (One Epoch of SGD)
24 print(f"{' '*60}")
25 print(f"STARTING SGD TRAINING: 1 EPOCH (4 SAMPLES)")
26 print(f"{' '*60}\n")
27
28 for i in range(len(X_train)):
29     # --- PREPARATION ---
30     # Grab one sample and reshape to column vector (2, 1)
31     x = X_train[i].reshape(2, 1)
32     target = y_train[i].reshape(1, 1)
33
34     print(f"--- STAGE {i+1}: Training on Input {X_train[i]} (Target: {y_train[i][0]}) ---")
35
36     # --- FORWARD PASS ---
37     # Layer 1 (Input to Hidden)
38     z1 = np.dot(W1, x) + b1
39     a1 = sigmoid(z1)
40
41     # Layer 2 (Hidden to Output)
42     z2 = np.dot(W2, a1) + b2
43     a2 = sigmoid(z2)
44
45     prediction = a2[0, 0]
46     error = target - a2
47     print(f" [Forward] Predicted: {prediction:.4f} | Error: {error[0, 0]:.4f}")
48
49     # --- BACKWARD PASS (The Chain Rule in action) ---
50
51     # 1. Calculate output layer gradient (delta)
52     # delta2 = error * derivative of sigmoid at output
53     delta2 = error * sigmoid_derivative(a2)
54
55     # 2. Calculate hidden layer gradient (delta)
56     # delta1 = (W2_transpose * delta2) * derivative of sigmoid at hidden
57     delta1 = np.dot(W2.T, delta2) * sigmoid_derivative(a1)
58
59     # --- WEIGHT UPDATES (Nudging the parameters) ---
60
61     # Update Layer 2
62     # W2_new = W2_old + (learning_rate * delta2 * a1_transpose)
63     W2_update = learning_rate * np.dot(delta2, a1.T)
64     W2 += W2_update
65     b2 += learning_rate * delta2
66
67     # Update Layer 1
68     # W1_new = W1_old + (learning_rate * delta1 * x_transpose)

```

```

69 W1_update = learning_rate * np.dot(delta1, x.T)
70 W1 += W1_update
71 b1 += learning_rate * delta1
72
73 print(f" [Backward] Updated W2 by average: {np.mean(np.abs(W2_update)):.6f}")
74 print(f" [Backward] Updated W1 by average: {np.mean(np.abs(W1_update)):.6f}\n")
75
76 print(f"{' '*60}")
77 print("End of Epoch 1. Weights have been adjusted 4 times.")

```

```

=====
STARTING SGD TRAINING: 1 EPOCH (4 SAMPLES)
=====

--- STAGE 1: Training on Input [0 0] (Target: 0) ---
[Forward] Predicted: 0.6381 | Error: -0.6381
[Backward] Updated W2 by average: 0.032546
[Backward] Updated W1 by average: 0.000000

--- STAGE 2: Training on Input [0 1] (Target: 1) ---
[Forward] Predicted: 0.6574 | Error: 0.3426
[Backward] Updated W2 by average: 0.022818
[Backward] Updated W1 by average: 0.004794

--- STAGE 3: Training on Input [1 0] (Target: 1) ---
[Forward] Predicted: 0.6967 | Error: 0.3033
[Backward] Updated W2 by average: 0.018607
[Backward] Updated W1 by average: 0.004551

--- STAGE 4: Training on Input [1 1] (Target: 0) ---
[Forward] Predicted: 0.7431 | Error: -0.7431
[Backward] Updated W2 by average: 0.050073
[Backward] Updated W1 by average: 0.016978

=====
End of Epoch 1. Weights have been adjusted 4 times.

```

The network is trained through 20000 epochs

```

1 epochs = 20000
2 learning_rate = 0.5
3 history = []
4
5 for epoch in range(epochs):
6     epoch_loss = 0
7     for i in range(len(X_train)):
8         # -- SGD: One sample at a time --
9         x = X_train[i].reshape(2, 1)
10        target = y_train[i].reshape(1, 1)
11
12        # Forward
13        a1 = sigmoid(np.dot(W1, x) + b1)
14        a2 = sigmoid(np.dot(W2, a1) + b2)
15
16        # Backward (using the logic from the previous script)
17        error = target - a2
18        delta2 = error * sigmoid_derivative(a2)
19        delta1 = np.dot(W2.T, delta2) * sigmoid_derivative(a1)
20
21        # Updates
22        W2 += learning_rate * np.dot(delta2, a1.T)
23        b2 += learning_rate * delta2
24        W1 += learning_rate * np.dot(delta1, x.T)
25        b1 += learning_rate * delta1
26
27        epoch_loss += np.mean(np.square(error))
28
29    history.append(epoch_loss / 4)
30
31    if epoch % 1000 == 0:
32        print(f"Epoch {epoch} - Loss: {history[-1]:.6f}")
33
34 print("\nTraining Complete!")

```

```

Epoch 0 - Loss: 0.283673
Epoch 1000 - Loss: 0.143743
Epoch 2000 - Loss: 0.136506
Epoch 3000 - Loss: 0.135075
Epoch 4000 - Loss: 0.134491
Epoch 5000 - Loss: 0.134178
Epoch 6000 - Loss: 0.133983
Epoch 7000 - Loss: 0.133851
Epoch 8000 - Loss: 0.133756

```

```
Epoch 9000 - Loss: 0.133684
Epoch 10000 - Loss: 0.133628
Epoch 11000 - Loss: 0.133583
Epoch 12000 - Loss: 0.133546
Epoch 13000 - Loss: 0.133515
Epoch 14000 - Loss: 0.133489
Epoch 15000 - Loss: 0.133467
Epoch 16000 - Loss: 0.133448
Epoch 17000 - Loss: 0.133431
Epoch 18000 - Loss: 0.133416
Epoch 19000 - Loss: 0.133403
```

Training Complete!

⌂ B I < > ↺ 📄 🔍 ⌵ ⌶ — Ψ ☺ 📄 Close

Now test the network for valid output. Output 2 is wrong so the network did not get trained properly.

Now test the network for valid output. Output 2 is wrong so the network did not get trained properly.

```
1 import numpy as np
2
3 # --- FINAL INFERENCE TEST ---
4 print(f"{' '*50}")
5 print("FINAL INFERENCE RESULTS (After Training)")
6 print(f"{' '*50}")
7 print(f"{'Input (X1, X2)':<18} | {'Target':<8} | {'Prediction (01)':<15}")
8 print("-" * 50)
9
10 # We use the weights (W1, W2) and biases (b1, b2)
11 # updated from your 10,000 epoch training loop
12 for i in range(len(X_train)):
13     # 1. Prepare the input sample
14     x_input = X_train[i].reshape(2, 1)
15     target_val = y_train[i][0]
16
17     # 2. Forward Pass: Layer 1
18     z1 = np.dot(W1, x_input) + b1
19     a1 = sigmoid(z1)
20
21     # 3. Forward Pass: Layer 2
22     z2 = np.dot(W2, a1) + b2
23     a2 = sigmoid(z2)
24
25     # 4. Final Result
26     prediction = a2[0, 0]
27
28     # Formatting the output for clarity
29     input_str = f"[{X_train[i][0]}, {X_train[i][1]}]"
30     print(f"{input_str:<18} | {target_val:<8} | {prediction:.6f}")
31
32 print(f"{' '*50}")
```

```
=====
FINAL INFERENCE RESULTS (After Training)
=====
Input (X1, X2)      | Target | Prediction (01)
-----
[0, 0]              | 0      | 0.012473
[0, 1]              | 1      | 0.483724
[1, 0]              | 1      | 0.986726
[1, 1]              | 0      | 0.484205
=====
```

We retrain the network by changing the hyperparameter learning rate = 1.0 and train the network for 20000 epochs

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # --- 1. Setup Data & Helper Functions ---
5 X_train = np.array([[0,0], [0,1], [1,0], [1,1]])
6 y_train = np.array([[0], [1], [1], [0]])
7
8 def sigmoid(x):
9     return 1 / (1 + np.exp(-x))
10
11 # --- 2. Initialize Network (2, 2, 1) ---
12 np.random.seed(7)
13 W1 = np.random.randn(2, 2)
```

```

14 b1 = np.random.randn(2, 1)
15 W2 = np.random.randn(1, 2)
16 b2 = np.random.randn(1, 1)
17
18 learning_rate = 1.0
19 epochs = 20000
20 history = []
21
22 print(f"Starting Training for {epochs} epochs...")
23
24 # --- 3. The Training Loop (SGD) ---
25 for epoch in range(epochs):
26     epoch_loss = 0 # Reset loss at the start of each epoch
27
28     for i in range(len(X_train)):
29         # Grab one sample (Stochastic)
30         x = X_train[i].reshape(2, 1)
31         target = y_train[i].reshape(1, 1)
32
33         # FORWARD PASS
34         # Layer 1 (Input to Hidden)
35         z1 = np.dot(W1, x) + b1
36         a1 = sigmoid(z1)
37
38         # Layer 2 (Hidden to Output)
39         z2 = np.dot(W2, a1) + b2
40         a2 = sigmoid(z2)
41
42         # CALCULATE ERROR (MSE contribution)
43         error = target - a2
44         epoch_loss += np.mean(np.square(error))
45
46         # BACKWARD PASS (Blame Assignment)
47         # Output Gradient
48         delta2 = error * (a2 * (1 - a2))
49         # Hidden Gradient
50         delta1 = np.dot(W2.T, delta2) * (a1 * (1 - a1))
51
52         # WEIGHT UPDATES (The Nudge)
53         W2 += learning_rate * np.dot(delta2, a1.T)
54         b2 += learning_rate * delta2
55         W1 += learning_rate * np.dot(delta1, x.T)
56         b1 += learning_rate * delta1
57
58     # Record average loss for this epoch
59     average_loss = epoch_loss / 4
60     history.append(average_loss)
61
62     # Print progress every 1000 epochs
63     if epoch % 1000 == 0:
64         print(f"Epoch {epoch:5d} | Mean Squared Error: {average_loss:.6f}")
65
66 print("\nTraining complete!")
67
68 # --- 4. Final Inference Test ---
69 print("\n" + "="*45)
70 print("FINAL INFERENCE RESULTS")
71 print("="*45)
72 for i in range(len(X_train)):
73     x_test = X_train[i].reshape(2, 1)
74     # Forward Pass Only
75     a1 = sigmoid(np.dot(W1, x_test) + b1)
76     a2 = sigmoid(np.dot(W2, a1) + b2)
77     print(f"Input: {X_train[i]} | Target: {y_train[i][0]} | Prediction: {a2[0,0]:.4f}")

```

```

Starting Training for 20000 epochs...
Epoch    0 | Mean Squared Error: 0.282076
Epoch 1000 | Mean Squared Error: 0.002604
Epoch 2000 | Mean Squared Error: 0.000890
Epoch 3000 | Mean Squared Error: 0.000527
Epoch 4000 | Mean Squared Error: 0.000372
Epoch 5000 | Mean Squared Error: 0.000287
Epoch 6000 | Mean Squared Error: 0.000233
Epoch 7000 | Mean Squared Error: 0.000196
Epoch 8000 | Mean Squared Error: 0.000169
Epoch 9000 | Mean Squared Error: 0.000149
Epoch 10000 | Mean Squared Error: 0.000132
Epoch 11000 | Mean Squared Error: 0.000119
Epoch 12000 | Mean Squared Error: 0.000109
Epoch 13000 | Mean Squared Error: 0.000100
Epoch 14000 | Mean Squared Error: 0.000092
Epoch 15000 | Mean Squared Error: 0.000086
Epoch 16000 | Mean Squared Error: 0.000080

```

```
Epoch 17000 | Mean Squared Error: 0.000075
Epoch 18000 | Mean Squared Error: 0.000071
Epoch 19000 | Mean Squared Error: 0.000067
```

Training complete!

FINAL INFERENCE RESULTS

```
Input: [0 0] | Target: 0 | Prediction: 0.0079
Input: [0 1] | Target: 1 | Prediction: 0.9909
Input: [1 0] | Target: 1 | Prediction: 0.9925
Input: [1 1] | Target: 0 | Prediction: 0.0071
```

We test the network again for all inputs. The output looks correct so we have a fully trained DNN that correctly implements the XOR gate

```
1 import numpy as np
2
3 # --- FINAL INFERENCE TEST ---
4 print(f"{' '*50}")
5 print("FINAL INFERENCE RESULTS (After Training)")
6 print(f"{' '*50}")
7 print(f"{'Input (X1, X2)':<18} | {'Target':<8} | {'Prediction (01)':<15}")
8 print("-" * 50)
9
10 # We use the weights (W1, W2) and biases (b1, b2)
11 # updated from your 10,000 epoch training loop
12 for i in range(len(X_train)):
13     # 1. Prepare the input sample
14     x_input = X_train[i].reshape(2, 1)
15     target_val = y_train[i][0]
16
17     # 2. Forward Pass: Layer 1
18     z1 = np.dot(W1, x_input) + b1
19     a1 = sigmoid(z1)
20
21     # 3. Forward Pass: Layer 2
22     z2 = np.dot(W2, a1) + b2
23     a2 = sigmoid(z2)
24
25     # 4. Final Result
26     prediction = a2[0, 0]
27
28     # Formatting the output for clarity
29     input_str = f"[{X_train[i][0]}, {X_train[i][1]}]"
30     print(f"{input_str:<18} | {target_val:<8} | {prediction:.6f}")
31
32 print(f"{' '*50}")
```

FINAL INFERENCE RESULTS (After Training)

```
Input (X1, X2) | Target | Prediction (01)
-----
[0, 0]         | 0      | 0.007945
[0, 1]         | 1      | 0.990898
[1, 0]         | 1      | 0.992513
[1, 1]         | 0      | 0.007108
```

We retrain the network using Batch Gradient Descent. That is we update the weight only once after cycling through entire dataset

```
1 import numpy as np
2
3 # --- 1. Data Setup (Transpose for Matrix Math) ---
4 # X_train is (4, 2), we want (2, 4) so each column is one XOR sample
5 X_batch = X_train.T
6 y_batch = y_train.T # (1, 4)
7
8 # --- 2. Re-Initialize ---
9 np.random.seed(7)
10 W1 = np.random.randn(2, 2)
11 b1 = np.random.randn(2, 1)
12 W2 = np.random.randn(1, 2)
13 b2 = np.random.randn(1, 1)
14
15 learning_rate = 1.0
16 epochs = 40000 # Batch often needs more epochs because updates are less frequent
17 history = []
18
19 # --- 3. Batch Training Loop ---
```



```

20 for epoch in range(epochs):
21     # FORWARD PASS (All 4 samples at once)
22     z1 = np.dot(w1, X_batch) + b1
23     a1 = sigmoid(z1)
24
25     z2 = np.dot(w2, a1) + b2
26     a2 = sigmoid(z2)
27
28     # ERROR CALCULATION
29     error = y_batch - a2
30     loss = np.mean(np.square(error))
31     history.append(loss)
32
33     # BACKWARD PASS (Batch Gradients)
34     delta2 = error * (a2 * (1 - a2))
35     # Note: We divide by 4 (the batch size) to get the average gradient
36     delta1 = np.dot(w2.T, delta2) * (a1 * (1 - a1))
37
38     # WEIGHT UPDATES (One update per epoch)
39     # Using dot product here effectively sums the gradients across the batch
40     w2 += (learning_rate / 4) * np.dot(delta2, a1.T)
41     b2 += (learning_rate / 4) * np.sum(delta2, axis=1, keepdims=True)
42
43     w1 += (learning_rate / 4) * np.dot(delta1, X_batch.T)
44     b1 += (learning_rate / 4) * np.sum(delta1, axis=1, keepdims=True)
45
46     if epoch % 5000 == 0:
47         print(f"Epoch {epoch:5d} | Batch Loss: {loss:.6f}")
48
49 print("\nBatch Training Complete!")

```

```

Epoch    0 | Batch Loss: 0.250642
Epoch 5000 | Batch Loss: 0.001927
Epoch 10000 | Batch Loss: 0.000684
Epoch 15000 | Batch Loss: 0.000409
Epoch 20000 | Batch Loss: 0.000291
Epoch 25000 | Batch Loss: 0.000225
Epoch 30000 | Batch Loss: 0.000183
Epoch 35000 | Batch Loss: 0.000154

```

Batch Training Complete!

```

1 import numpy as np
2
3 # --- 1. SETUP: Input Image with a Vertical Bar ---
4 image = np.zeros((10, 10))
5 image[:, 4:6] = 10
6
7 # Define Two Kernels
8 v_kernel = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]]) # Vertical Edge
9 h_kernel = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]]) # Horizontal Edge
10
11 # --- 2. CONVOLUTION LAYER ---
12 def conv2d(img, k):
13     h, w = img.shape
14     kh, kw = k.shape
15     res = np.zeros((h-kh+1, w-kw+1))
16     for r in range(h-kh+1):
17         for c in range(w-kw+1):
18             res[r, c] = np.sum(img[r:r+kh, c:c+kw] * k)
19     return res
20
21 v_feature_map = conv2d(image, v_kernel)
22 h_feature_map = conv2d(image, h_kernel)
23
24 # --- 3. MAX POOLING (2x2 Stride 2) ---
25 def max_pooling(feature_map, size=2):
26     h, w = feature_map.shape
27     res = np.zeros((h // size, w // size))
28     for r in range(0, h, size):
29         for c in range(0, w, size):
30             # Take the maximum value in the 2x2 window
31             window = feature_map[r:r+size, c:c+size]
32             res[r // size, c // size] = np.max(window)
33     return res
34
35 v_pool = max_pooling(v_feature_map)
36 h_pool = max_pooling(h_feature_map)
37
38 # --- 4. PRINTING THE HARDWARE PIPELINE ---
39 print("*60")
40 print("THE CNN PIPELINE: VERTICAL vs HORIZONTAL")

```

```

41 print("="*60)
42
43 print("\n[STEP 1] 8x8 VERTICAL FEATURE MAP (After Conv):")
44 for row in v_feature_map:
45     print(" ".join(f"{int(val):3d}" for val in row))
46
47 print("\n[STEP 2] 4x4 MAX POOLED MAP (Data Compression):")
48 for row in v_pool:
49     print(" ".join(f"{int(val):3d}" for val in row))
50
51 # --- 5. SIMPLE CLASSIFIER LOGIC ---
52 v_score = np.sum(v_pool)
53 h_score = np.sum(h_pool)
54
55 print("\n" + "="*60)
56 print("FINAL CLASSIFICATION")
57 print("="*60)
58 print(f"Vertical Signal Strength: {v_score}")
59 print(f"Horizontal Signal Strength: {h_score}")
60
61 if v_score > h_score and v_score > 0:
62     print("\nRESULT: VERTICAL EDGE DETECTED!")
63 elif h_score > v_score and h_score > 0:
64     print("\nRESULT: HORIZONTAL EDGE DETECTED!")
65 else:
66     print("\nRESULT: NO CLEAR EDGE DETECTED")

```

```

=====
THE CNN PIPELINE: VERTICAL vs HORIZONTAL
=====

```

```
[STEP 1] 8x8 VERTICAL FEATURE MAP (After Conv):
```

```

0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0
0  0  30  30 -30 -30  0  0

```

```
[STEP 2] 4x4 MAX POOLED MAP (Data Compression):
```

```

0  30 -30  0
0  30 -30  0
0  30 -30  0
0  30 -30  0

```

```
=====
FINAL CLASSIFICATION
=====
```

```

Vertical Signal Strength:  0.0
Horizontal Signal Strength: 0.0

```

```
RESULT: NO CLEAR EDGE DETECTED
```

Test if the DNN is trained correctly. It is

```

1 import numpy as np
2
3 # --- FINAL INFERENCE TEST ---
4 print(f"{' '*50}")
5 print("FINAL INFERENCE RESULTS (After Training)")
6 print(f"{' '*50}")
7 print(f"{'Input (X1, X2)':<18} | {'Target':<8} | {'Prediction (01)':<15}")
8 print("-" * 50)
9
10 # We use the weights (w1, w2) and biases (b1, b2)
11 # updated from your 10,000 epoch training loop
12 for i in range(len(X_train)):
13     # 1. Prepare the input sample
14     x_input = X_train[i].reshape(2, 1)
15     target_val = y_train[i][0]
16
17     # 2. Forward Pass: Layer 1
18     z1 = np.dot(w1, x_input) + b1
19     a1 = sigmoid(z1)
20
21     # 3. Forward Pass: Layer 2
22     z2 = np.dot(w2, a1) + b2
23     a2 = sigmoid(z2)
24
25     # 4. Final Result

```

```
26 prediction = a2[0, 0]
27
28 # Formatting the output for clarity
29 input_str = f"[{X_train[i][0]}, {X_train[i][1]}]"
30 print(f"{input_str:<18} | {target_val:<8} | {prediction:.6f}")
31
32 print(f"{' '*50}")
```

```
=====
FINAL INFERENCE RESULTS (After Training)
=====
Input (X1, X2)    | Target | Prediction (O1)
-----
[0, 0]            | 0      | 0.011560
[0, 1]            | 1      | 0.986805
[1, 0]            | 1      | 0.989128
[1, 1]            | 0      | 0.010298
=====
```

1 Start coding or [generate](#) with AI.